**SIMATS ENGINEERING**

**ASSESSMENT TOOL 1**

**COURSE NAME**: **COMPUTER ARCHITECTURE**

COURSE CODE: CSA1204

**COURSE OUTCOME COVERED CO1.** Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)

Assessment Tool Weightage: 40%

QUESTIONS:5 Total Marks:25

ANNEXURE A

APPLICATION BASED QUESTIONS

Code of Conduct:

I _YUVASRI.S(192521032)_____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

**ANNEXURE A**

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.

ANSWER :

In a **smart city surveillance system**, multiple cameras continuously send video data to the computer. The **I/O devices (cameras)** transfer this data to **memory (RAM)**, and the **CPU** fetches the data from memory to analyze it and detect threats. During **peak hours**, many cameras transmit data simultaneously, causing heavy traffic between the CPU, memory, and I/O devices. This results in **bus congestion**, increased memory access time, and CPU waiting for data, leading to **lag and delayed threat detection**.

A **single-bus structure** uses one common bus for communication between the CPU, memory, and I/O devices. Since all components share the same bus, only one data transfer can occur at a time, creating a bottleneck and reducing overall performance. In contrast, a **multi-bus structure** uses separate buses for memory and I/O communication, allowing multiple data transfers simultaneously. This reduces bus contention, improves data throughput, and enables faster processing of video streams.

The **instruction execution cycle** can also be improved to enhance performance. Techniques such as **instruction pipelining** allow multiple instructions to be processed at different stages simultaneously, reducing execution time. **Cache memory** stores frequently used data close to the CPU, minimizing memory access delays. **Direct Memory Access (DMA)** enables I/O devices to transfer data directly to memory without continuous CPU involvement, reducing CPU workload. Faster processors, multicore CPUs, and optimized instruction scheduling further improve response time and increase the efficiency of real-time surveillance systems.

**Conclusion:** By using a **multi-bus architecture**, **cache memory**, **DMA**, and **instruction pipelining**, the surveillance system can process large volumes of video data more efficiently, reducing lag and improving real-time threat detection.

2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.

ANSWER :

In a **real-time stock trading application**, the processor executes millions of instructions every second to process buy and sell transactions. Every instruction passes through the **Fetch–Decode–Execute cycle**. During the **Fetch** stage, the CPU retrieves instructions from memory. In the **Decode** stage, it interprets the instruction, and in the **Execute** stage, it performs the required operation. Under high load, frequent **memory accesses** and **branch instructions** increase the number of clock cycles per instruction (**CPI**). Memory delays cause the CPU to wait for data, while branch instructions may lead to pipeline stalls and incorrect instruction fetching. As a result, execution time increases, causing delays in transaction processing.

To reduce **CPI** and improve CPU performance, several architectural enhancements can be used. **Cache memory (L1, L2, L3)** reduces memory access time by storing frequently used data close to the CPU. **Instruction pipelining** allows multiple instructions to be processed simultaneously at different stages, improving throughput. **Branch prediction** minimizes delays caused by branch instructions by predicting the next instruction accurately. **Out-of-order execution** enables the CPU to execute independent instructions while waiting for memory operations. **Superscalar processors** execute multiple instructions in a single clock cycle,

and **multicore processors** distribute transactions across multiple cores for parallel processing. Faster memory systems and optimized instruction scheduling also help reduce execution time.

**Conclusion**

By using **cache memory, instruction pipelining, branch prediction, out-of-order execution, superscalar architecture, and multicore processors**, the system can significantly reduce CPI, improve CPU utilization, increase instruction throughput, and ensure faster and more reliable real-time stock transaction processing during high workloads.

3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

ANSWER :

In an **8085-based industrial temperature control system**, temperature sensors provide input data to the microprocessor, and the processed results are sent to actuators to control heating or cooling. The **addressing mode** determines how the 8085 accesses data or memory during program execution. Incorrect use of addressing modes can cause the processor to read the wrong memory location or incorrect sensor values, leading to inaccurate temperature readings and improper actuator responses.

The **Immediate Addressing Mode** uses data directly in the instruction and is suitable for fixed values. The **Register Addressing Mode** accesses data stored in CPU registers, providing faster execution. The **Direct Addressing Mode** accesses data from a specified memory address, making it useful for sensor data stored in memory. The **Register Indirect Addressing Mode** uses register pairs (HL, BC, or DE) to point to memory locations, allowing flexible access to sensor readings. The **Implied Addressing Mode** operates on predefined registers without explicitly specifying the operand.

Possible causes of incorrect temperature readings include using the wrong addressing mode, incorrect memory addresses, improper initialization of registers, data corruption during memory access, programming errors, and failure to validate sensor input. These issues may cause the CPU to process invalid data, resulting in incorrect temperature control.

The **8085 instruction set** can improve system reliability by using **MOV** for correct data transfer, **LDA** and **STA** for accurate memory access, **MVI** for loading constants, **LXI** for initializing register pairs, **CMP** for comparing sensor values with threshold limits, **JZ**, **JC**, and **JNZ** for conditional decision-making, and **IN** and **OUT** instructions for communication with sensors and actuators. Proper use of these instructions, along with correct addressing modes, ensures accurate data processing and reliable control.

**Conclusion**

By selecting the appropriate **addressing modes**, correctly initializing registers and memory, and using the **8085 instruction set** effectively, the embedded temperature control system can process sensor data accurately, reduce errors, and provide reliable industrial temperature regulation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.

ANSWER :

In an 8086 microprocessor, memory is organized using **segmentation**, which divides the 1 MB memory space into **Code Segment (CS), Data Segment (DS), Stack Segment (SS), and Extra Segment (ES)**. The **Code Segment (CS)** stores program instructions, the **Data Segment (DS)** stores variables and data, the **Stack Segment (SS)** stores temporary data, function calls, and return addresses, and the **Extra Segment (ES)** is used for additional data operations. Each segment works with an offset address to form the physical memory address.

Improper handling of segments can lead to several problems. If the **Data Segment (DS)** points to an incorrect memory location, the processor may read or write invalid data, causing incorrect program output. If the **Stack Segment (SS)** or **Stack Pointer (SP)** is not initialized correctly, push and pop operations may access invalid memory, resulting in **stack overflow** or stack corruption. Incorrect use of the **Code Segment (CS)** may cause the CPU to fetch the wrong instructions, leading to program crashes or unexpected behavior.

To ensure correct program execution, the fetch–decode–execute cycle must access the correct segment and offset for every instruction. Appropriate addressing modes should also be used. Immediate addressing is suitable for constants**,** Register addressing provides fast data access**,** Direct addressing accesses fixed memory locations**,** Register Indirect addressing accesses memory through registers, and Based/Indexed addressing efficiently accesses arrays and data structures. Proper initialization of CS, DS, SS, ES, careful stack management, and correct use of addressing modes help prevent memory access errors.

**Conclusion**

By correctly managing code, data, and stack segments, properly initializing segment registers, maintaining the stack, and using suitable addressing modes, the 8086 microprocessor can execute multiple programs reliably, avoid incorrect data access and stack overflow errors, and improve overall system performance.

5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

ANSWER :

In a data communication system, packet information is often represented in hexadecimal because it is compact, easy to read, and directly corresponds to binary values used by the CPU. Since computers process data in binary**,** hexadecimal values must be accurately converted into binary

for correct instruction execution and data processing. For example, the hexadecimal value 3A is equivalent to the binary value 00111010**.** Accurate conversion ensures that packet headers, addresses, and control information are interpreted correctly during transmission and processing.

Errors in hexadecimal-to-binary conversion can lead to incorrect instruction execution and data processing. If a software engineer misinterprets a hexadecimal value, the CPU may execute the wrong instruction, access an incorrect memory location, or process invalid packet information. Such errors can result in corrupted data, communication failures, incorrect packet routing, system crashes, or reduced reliability in real-time applications.

An efficient CPU design helps minimize these problems by using cache memory to reduce memory access time, instruction pipelining to process multiple instructions simultaneously, and error detection mechanisms to identify invalid data before execution**.** Benchmarking evaluates CPU performance using metrics such as execution time, throughput, CPI (Cycles Per Instruction), and MIPS (Million Instructions Per Second)**.** Regular benchmarking helps detect performance bottlenecks, verify correct instruction execution, and improve system reliability under real-time workloads.

## Conclusion

Accurate hexadecimal-to-binary conversion is essential for correct packet processing and instruction execution in data communication systems. Efficient CPU architecture, combined with benchmarking techniques, improves processing speed, detects conversion-related errors, and ensures reliable performance in real-time communication systems.